◎ LARAVEL — TP PRATIQUE

Mini TP guidé : CRUD Produits

Exercice court en 3 étapes : lisez l'énoncé, codez, puis révélez la correction.

1 — 2 — 3

🗄️ ÉTAPE 1

La Migration

✎ ÉNONCÉ

Créer la structure de la table products en base de données

`database/migrations`

📖 Dans Laravel, une **migration** est un fichier PHP qui décrit la structure d'une table. C'est comme un "plan d'architecte" pour votre base de données. Vous devez créer la table `products` qui stockera nos produits.

```
$ php artisan make:migration create_products_table
```

Copier

🎯 OBJECTIFS

- [ ] Générer le fichier de migration avec **Artisan**
- [ ] Ajouter un champ `name` de type **string**
- [ ] Ajouter un champ `description` de type **text** (nullable)
- [ ] Ajouter un champ `price` de type **decimal** avec 2 décimales
- [ ] Exécuter `php artisan migrate` pour appliquer

LARAVEL — TP PRATIQUE

Mini TP guidé : CRUD Produits

Exercice court en 3 étapes : lisez l'énoncé, codez, puis révélez la correction.

1 — 2 — 3

ÉTAPE 1

La Migration

Créer la structure de la table products en base de données

`database/migrations`

CORRECTION

Dans Laravel, une **migration** est un fichier PHP qui décrit la structure d'une table. C'est comme un "plan d'architecte" pour votre base de données. Vous devez créer la table `products` qui stockera nos produits.

```
$ php artisan make:migration create_products_table
```

Copier

OBJECTIFS

- [ ] Générer le fichier de migration avec **Artisan**
- [ ] Ajouter un champ `name` de type **string**
- [ ] Ajouter un champ `description` de type **text** (nullable)
- [ ] Ajouter un champ `price` de type **decimal** avec 2 décimales
- [ ] Exécuter `php artisan migrate` pour appliquer

LARAVEL — TP PRATIQUE

Mini TP guidé : CRUD Produits

Exercice court en 3 étapes : lisez l'énoncé, codez, puis révélez la correction.

1 — 2 — 3

ÉTAPE 2

Le Modèle Eloquent

Créer la classe Product et déclarer les champs autorisés

ÉNONCÉ

app/Models/Product.php

Le **modèle** est la représentation PHP de votre table en base de données. Il agit comme un pont entre votre code et la BD. La propriété `$fillable` est une liste blanche qui protège contre l'**assignation massive** (Mass Assignment Attack).

```
$ php artisan make:model Product
```

Copier

OBJECTIFS

- [ ] Générer le modèle avec **Artisan**
- [ ] Déclarer la propriété `$fillable` avec les 3 champs autorisés
- [ ] Ajouter un cast pour que `price` soit traité comme un **décimal**

INDICES

C'est quoi $fillable ?

LARAVEL — TP PRATIQUE

Mini TP guidé : CRUD Produits

Exercice court en 3 étapes : lisez l'énoncé, codez, puis révélez la correction.

1 — 2 — 3

ÉTAPE 2

Le Modèle Eloquent

Créer la classe Product et déclarer les champs autorisés

`app/Models/Product.php`

CORRECTION

Le **modèle** est la représentation PHP de votre table en base de données. Il agit comme un pont entre votre code et la BD. La propriété `$fillable` est une liste blanche qui protège contre l'**assignation massive** (Mass Assignment Attack).

```
$ php artisan make:model Product
```

Copier

OBJECTIFS

- [ ] Générer le modèle avec **Artisan**
- [ ] Déclarer la propriété `$fillable` avec les 3 champs autorisés
- [ ] Ajouter un cast pour que `price` soit traité comme un **décimal**

INDICES

C'est quoi $fillable ?

LARAVEL — TP PRATIQUE

Mini TP guidé : CRUD Produits

Exercice court en 3 étapes : lisez l'énoncé, codez, puis révélez la correction.

1 — 2 — 3

ÉTAPE 3

Le Contrôleur

Implémenter index(), create(), store() et destroy()

`app/Http/Controllers`

ÉNONCÉ

Le **contrôleur** est le chef d'orchestre de votre application. Il reçoit les requêtes HTTP, interroge le modèle, et renvoie la bonne vue. Vous allez implémenter 4 méthodes : afficher la liste, afficher le formulaire, enregistrer un produit, et le supprimer.

```
$ php artisan make:controller ProductController --resource
```

Copier

OBJECTIFS

- Générer le contrôleur resource avec **Artisan**
- `index()` — récupérer tous les produits paginés et retourner la vue
- `create()` — retourner simplement la vue du formulaire de création
- `store()` — valider la requête, créer le produit, rediriger avec message
- `destroy()` — supprimer le produit et rediriger avec message

LARAVEL — TP PRATIQUE

Mini TP guidé : CRUD Produits

Exercice court en 3 étapes : lisez l'énoncé, codez, puis révélez la correction.

1 — 2 — 3

ÉTAPE 3

Le Contrôleur

Implémenter index(), create(), store() et destroy()

app/Http/Controllers

CORRECTION

Le **contrôleur** est le chef d'orchestre de votre application. Il reçoit les requêtes HTTP, interroge le modèle, et renvoie la bonne vue. Vous allez implémenter 4 méthodes : afficher la liste, afficher le formulaire, enregistrer un produit, et le supprimer.

```
$ php artisan make:controller ProductController --resource
```

Copier

OBJECTIFS

- Générer le contrôleur resource avec **Artisan**
- `index()` — récupérer tous les produits paginés et retourner la vue
- `create()` — retourner simplement la vue du formulaire de création
- `store()` — valider la requête, créer le produit, rediriger avec message
- `destroy()` — supprimer le produit et rediriger avec message